

UC MATH 600 MATHEMATICS FOR COMPUTER SCIENCE · MODULE 05 · LECTURE NOTES

Number Theory and Modular Arithmetic

The arithmetic every hash, cipher, and index uses.

MODULE OUTCOMES

- Use the division algorithm and gcd fluently.
- Run the Euclidean algorithm, extended and not.
- Compute in modular arithmetic, including inverses.
- State and use Fermat's little theorem.
- Explain where this is used in the program.

1 Divisibility and gcd

DIVISION ALGORITHM

for any a and any $b > 0$ there are UNIQUE q, r with
 $a = qb + r$ and $0 \leq r < b$
 Unique. That uniqueness is exactly what makes
 " $a \bmod b$ " well-defined (Module 04 section 2).

EUCLID'S ALGORITHM

$\gcd(a, b) = \gcd(b, a \bmod b)$, $\gcd(a, 0) = a$

why it works: any common divisor of a and b also divides $a - qb = r$, and conversely any common divisor of b and r divides a . So the pair (b, r) has exactly the same common divisors as (a, b)
 -- and the numbers strictly shrink.

it runs in $O(\log \min(a, b))$ steps, which is why
 it is still in use after two thousand years

EXTENDED EUCLID

also returns x, y with $ax + by = \gcd(a, b)$
 -- Bezout's identity -- and that x is how you
 compute modular inverses (section 3)

Extended Euclid gives you modular inverses, which is the whole reason it matters for cryptography: if $ax + ny = 1$ then $ax \equiv 1 \pmod{n}$, so x is a^{-1} . **The uniqueness clause in the division algorithm is what makes everything later well-defined** — without it, ' $a \bmod b$ ' would not name a single number, and §2's whole construction would collapse.

2 Modular arithmetic

$A \equiv B \pmod{N}$ MEANS N DIVIDES $A - B$, AND THIS IS AN EQUIVALENCE RELATION

It is reflexive, symmetric, and transitive — each takes one line to check — **so it partitions the integers into n residue classes** (Module 04 §3's equivalence-equals-partition correspondence, in its first serious application).

And addition and multiplication are well-defined on those classes — **which is the thing that actually has to be proved**, and **which Module 04 §2 warned about**: if $a \equiv a'$ and $b \equiv b'$, you must show $ab \equiv a'b'$, or computing with representatives is not legitimate.

Once proved, you can compute with remainders freely, reducing at any point in a calculation — **which is what makes modular exponentiation of enormous numbers tractable**: you never hold the full value, only its residue (repeated squaring gives $a^k \bmod n$ in $O(\log k)$ multiplications).

But division is not automatic: a has a multiplicative inverse modulo n exactly when $\gcd(a, n) = 1$ — and **extended Euclid computes it** (§1), which is the practical payoff of the whole section.

3 The theorems

Fermat: if p is prime and p does not divide a , then $a^{p-1} \equiv 1 \pmod{p}$

Euler: $a^{\phi(n)} \equiv 1 \pmod{n}$ whenever $\gcd(a, n) = 1$

- **Fermat's little theorem** gives you inverses instantly modulo a prime (since $a \cdot a^{p-2} \equiv 1$) and underlies the standard probabilistic primality tests — which is CSCE 658's territory.
- **Euler's theorem** is the generalisation to composite n , where $\phi(n)$ counts the integers below n that are coprime to it — **and RSA is this theorem**: encryption raises to e , decryption raises to d , and $ed \equiv 1 \pmod{\phi(n)}$ makes the round trip the identity.
- **The Chinese remainder theorem** says congruences modulo coprime moduli have a unique joint solution — which lets you work modulo the factors separately and recombine, and is used both for speed (RSA decryption is several times faster this way) and in secret sharing schemes.
- **RSA is Euler's theorem** — which is why CSCE 711 can treat it in a single module rather than five, and is a good illustration of how much a prerequisite buys. **These three theorems are the whole number-theoretic toolkit this program needs.**

4 Where it is used

- **Hashing** — every hash table's index is a mod operation, and the choice of modulus determines the collision behaviour (a power of two throws away the high bits; a prime does not) — CSCE 629 and CSCE 658.
- **Public-key cryptography** — RSA is Euler's theorem and Diffie-Hellman is modular exponentiation in a group (CSCE 711) — and Shor's algorithm attacks exactly the order-finding problem of this module (CSCE 640 Module 08 §1).
- **Checksums and error detection** — CRC is polynomial arithmetic modulo a fixed polynomial, which is structurally the same construction over a different ring.
- **Random number generation** — linear congruential generators are one mod away from being useful, and understanding exactly why they are weak requires this module (the low bits have short periods).
- **And graphics, less obviously** — wrapping texture coordinates, tiling patterns, and hash-based procedural noise are all modular arithmetic. Hash-based procedural noise is the one place this module touches the graphics track directly, and it touches it constantly.

AND THE THING TO CARRY FORWARD

Modular arithmetic is the first place Module 04's well-definedness obligation becomes real — computing with a representative and then claiming an answer about the whole class is exactly the move that needs justifying, and here it is justified rather than assumed.

And the Euclidean algorithm is the first non-trivial algorithm in the program — with a correctness argument and a complexity bound, which is precisely what CSCE 629 is about and is a preview of its method.

So prove Euclid correct and bound its running time by hand — which is a complete exercise in everything Modules 01 through 03 taught: a precise statement, an invariant, an induction, and an asymptotic claim.

Which is why this module sits here rather than later: it is the point at which the proof technique first visibly pays for itself, on an object you already understood.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Lehman, Leighton & Meyer, chapters 8 and 9 (free)	The whole module , free — and it develops RSA from this material, which is the right motivation.
MIT 6.042J, the number theory lectures (free video)	§§1 to 3 , with Euclid proved rather than asserted.
Silverman — A Friendly Introduction to Number Theory	§§1 to 3 — patient, and the early chapters are available free from the author.
Project Euler, the early problems (free)	§§1 and 4 — problems where implementing Euclid and modular exponentiation is the natural solution.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **State the division algorithm** and prove q and r are unique.
- 2 **Run Euclid by hand** on three pairs, showing every step.
- 3 **Prove Euclid correct** using the common-divisor argument.
- 4 **Bound its running time** and justify the log.

- 5 **Implement extended Euclid** and verify Bezout's identity.
- 6 **Prove that addition is well-defined** on residue classes.
- 7 **Compute five modular inverses**, and find one that does not exist.
- 8 **Implement modular exponentiation** by repeated squaring.
- 9 **Verify Fermat's little theorem** on several primes, then on a composite.
- 10 **Work a small RSA example** end to end by hand.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 State the division algorithm and say what the uniqueness buys.
- 2 Why does Euclid's algorithm work?
- 3 What is its running time, and why?
- 4 What does extended Euclid give you beyond the gcd?
- 5 Show congruence is an equivalence relation.
- 6 What has to be proved before computing with representatives?
- 7 When does a have an inverse mod n ?
- 8 State Fermat, Euler, and the Chinese remainder theorem.
- 9 Which theorem is RSA?
- 10 Name five places this arithmetic is used in the program.